

Do For hbase now same we did like only

Here is the complete step-by-step procedure for the **HBase Lab Use Case**, structured in the exact same clear, terminal-based format as the Hive lab.

TXT

All system-dependent values (such as paths, table names, column families, row keys, and column attributes) that need to be updated when systems or use cases change are enclosed in `<< >>`.

Group A: System & Service Initialization (Linux Shell)

Terminal 1: Environment Check & Services Start

1. Verify HBase Installation & Environment:

Bash



```
hbase version
echo $HBASE_HOME[cite: 2]
which hbase[cite: 2]
```

1. Start Hadoop & HBase Services:

Bash



```
start-dfs.sh[cite: 2, 3]
start-yarn.sh[cite: 2, 3]
start-hbase.sh[cite: 2]
```

1. Verify Active Java Processes:

Bash



```
jps[cite: 2, 3]
```

(Ensure `HMaster`, `HRegionServer`, `NameNode`, and `DataNode` are running in the process list)[cite: 2].

Group B: HBase Operations & Data Manipulation (HBase Shell)

Terminal 1 (or New Terminal 2): HBase Interactive Shell

1. Launch HBase Shell:

Bash



```
hbase shell[cite: 2]
```

1. Create Table with Column Families:

Ruby



```
create '<<customer_profiles>>', '<<personal>>', '<<service>>'[cite: 2]
```

1. Verify Table Structure & List Tables:

Ruby



```
list[cite: 2]  
describe '<<customer_profiles>>'[cite: 2]
```

1. Insert / Put Data Records (Row Keys & Column Values):

Ruby



```
# Insert Record 1 (CUST1001)
put '<<customer_profiles>>', '<<CUST1001>>', '<<personal>>:<<name>>', '<<Rahul Sharma>>'
put '<<customer_profiles>>', '<<CUST1001>>', '<<personal>>:<<city>>', '<<Bengaluru>>'
put '<<customer_profiles>>', '<<CUST1001>>', '<<personal>>:<<mobile>>', '<<9876543210>>'
put '<<customer_profiles>>', '<<CUST1001>>', '<<service>>:<<plan>>', '<<5G Premium>>'
put '<<customer_profiles>>', '<<CUST1001>>', '<<service>>:<<status>>', '<<Active>>'[c
put '<<customer_profiles>>', '<<CUST1001>>', '<<service>>:<<data_usage>>', '<<42GB>>'

# Insert Record 2 (CUST1002)
put '<<customer_profiles>>', '<<CUST1002>>', '<<personal>>:<<name>>', '<<Priya Nair>>'
put '<<customer_profiles>>', '<<CUST1002>>', '<<personal>>:<<city>>', '<<Chennai>>'[c
put '<<customer_profiles>>', '<<CUST1002>>', '<<service>>:<<plan>>', '<<4G Standard>>'

# Insert Record 3 (CUST1003)
put '<<customer_profiles>>', '<<CUST1003>>', '<<personal>>:<<name>>', '<<Arun Kumar>>'
put '<<customer_profiles>>', '<<CUST1003>>', '<<service>>:<<plan>>', '<<5G Basic>>'[c
```

1. Retrieve Single Row Data (Get):

Ruby



```
get '<<customer_profiles>>', '<<CUST1001>>'[cite: 2]
```

1. Scan Full Table & Count Rows:

Ruby



```
scan '<<customer_profiles>>'[cite: 2]
count '<<customer_profiles>>'[cite: 2]
```

1. Update Existing Cell Value (Overwrites/Versions):

Ruby



```
put '<<customer_profiles>>', '<<CUST1001>>', '<<service>>:<<plan>>', '<<5G Unlimited>>'
get '<<customer_profiles>>', '<<CUST1001>>'[cite: 2]
```

1. Delete Specific Column Value:

Ruby



```
delete '<<customer_profiles>>', '<<CUST1001>>', '<<personal>>:<<mobile>>'[cite: 2]
get '<<customer_profiles>>', '<<CUST1001>>'[cite: 2]
```

1. Delete Entire Row:

Ruby



```
deleteall '<<customer_profiles>>', '<<CUST1003>>'[cite: 2]
get '<<customer_profiles>>', '<<CUST1003>>'[cite: 2]
count '<<customer_profiles>>'[cite: 2]
```

1. Disable and Drop Table (Cleanup / Reset Workflow):

Ruby



```
disable '<<customer_profiles>>'
drop '<<customer_profiles>>'
```

Summary Flow for Exam Memory

Plaintext



```
start-hbase.sh ↓ hbase shell ↓ create 'table', 'cf1', 'cf2' ↓ describe ↓ put (insert/
``[cite: 2]
```

